Kalm Language Specification

VERSION	1.1
STATUS	DRAFT
DATE	2026-10-10
JAVA VERSION	25



Contents

- 1. The shape of Kalm
  - A small taste
- 2. Source files and lexical rules
  - Tokens, whitespace, and comments
- 3. Types, values, and Java mapping
- 4. Bindings, expressions, and functions
- 5. Flow, lambdas, and errors
  - Matching and checks
  - Resource and scope management
  - Tasks and synchronization
- 6. Classes, contracts, and visibility
- 7. Packages and Java interoperability
- 8. Standard library
- 9. Compiler and Java 25 backend
- 10. Maven and Gradle
  - Maven
  - Gradle
- 11. Editor support
- 12. Core grammar
- 13. Diagnostics, conformance, and delivery gates
- 14. Implementation plan
- 15. Deferred from 1.x
- 16. First vertical slice
- References

1. The shape of Kalm

Kalm is a statically typed, object-oriented JVM language. It has Java's class model, predictable execution, explicit types at API boundaries, and direct access to Java libraries. Everyday syntax uses Java-style braces, Kalm keywords, inferred local types, and concise expression-bodied functions. Blocks use either braces or a colon followed by an inline or indented body. Simple statements end with a semicolon or a logical newline; semicolons are recommended.

Area	Kalm 1.x contract
Runtime	JVM; JDK 25 or newer to compile, Java 25 or newer to run
Compiler	Java compiler application that parses and checks Kalm, generates Java source, then invokes `javac`
Source	UTF-8 `.kalm`, Interchangeable `{}` and colon blocks; recommended, optional semicolons

Interop	Java types, methods, constructors, generics, arrays, exceptions, and dependencies on the project classpath/module path
Builds	CLI, Maven plugin, and Gradle plugin share one compiler core
Editors	VS Code extension and IntelliJ IDEA plugin backed by the Kalm language server
Runtime library	Small support jar; the JDK and project dependencies provide the rest

Kalm does not bundle Java libraries. Any library available to the host Java project is available to Kalm, subject to normal Java visibility, classpath, module readability, and module export rules.

A small taste

KALM

```
space studio.greetings;
use java.time.LocalDate;
use java.util.ArrayList;

pub class Greeter {
    fun greet(name: Text) -> Text = @"Hey, $name. Glad you're here.";
}

pub fun main(args: Array<Text>) -> Unit {
    fix today = LocalDate.now();
    fix names = ArrayList<Text>();
    names.add("Tim");
    sayLine(Greeter().greet(names.get(0)));
}
```

Kalm keeps the Java object model and library ecosystem, with concise function syntax and inferred local types. Braces and recommended semicolons keep statement structure familiar to Java developers; colon blocks and logical newlines offer an alternative, and inferred local types and expression-bodied functions trim routine syntax.

2. Source files and lexical rules

A file contains an optional `space` declaration, zero or more `use` declarations, then declarations. Top-level executable statements are not allowed. A source file may contain at most one `pub fun main(args: Array<Text>) -> Unit` or zero-argument `pub fun main() -> Unit` entry point. The compiler must reject multiple selected entry points for one application. The compiler generates the JVM-required `public static void main(String[] args)` launcher; for the argument-taking form, the Kalm entry point receives the original Java `String[]` as `Array<Text>` without copying. A compiler gives each source file with top-level definitions a deterministic Java facade named `<SourceBase>Kalm`. Public top-level classes and contracts live in same-named `.kalm` files.

Source paths follow the host Java project's source roots. If `space` is omitted, the file uses the unnamed package. A filename that cannot produce a unique Java identifier for its facade is a compile error.

Tokens, whitespace, and comments

Identifiers are case-sensitive, start with a Unicode letter or underscore, and continue with Unicode letters, digits,

or underscores. The lexer recognizes multi-character operators before their prefixes using longest-match tokenization. @ introduces an annotation use.

Simple statements terminate with `;`, a logical newline, or immediately before the end of a block or file. Separate simple statements on the same physical line require `;`. Semicolons are recommended but optional. A completed `return` followed by a logical newline is a bare return. Expressions continue inside parentheses, indexing, collection literals, and generic type arguments, and after an unfinished operator, comma, or member-access operator.

Every block body may use `{ ... }` or `: ...`. A colon followed by a newline requires a nonempty indented body, terminated by dedentation. Blank and comment-only lines do not affect indentation. Tabs expand to eight-column stops; ambiguous tab/space indentation and inconsistent dedentation are errors. An inline colon suite contains simple statements or declarations separated by semicolons and ends at its logical newline. Nested compound bodies require a multiline suite. Empty bodies use `{}`. Brace blocks and colon blocks may nest freely; indentation is structural only in colon suites. Literal delimiters retain their existing grammar.

Control-flow headers accept either parenthesized or unparenthesized conditions and loop bindings. Colon-bodied lambdas use parenthesized parameters to distinguish the body colon from parameter type annotations. For example:

```kalm
pub fun main() -> Unit {
    fix ready = true
    if ready:
        sayLine("Ready");
    else {
        sayLine("Waiting")
    }
}
```

The 1.0 keyword `fixed` is rejected with a migration diagnostic suggesting `fix`.

Comments use # to end a line and `## ... ##` for a block comment. Documentation comments begin with ### and continue to the end of that line. The lexer recognizes ### first, then the two-character block opener ##, then the single-character line comment #. Block comments do not nest. Documentation comments are preserved as generated Java documentation where practical.

Reserved words are: `space`, `use`, `class`, `contract`, `fun`, `var`, `fix`, `pub`, `internal`, `private`, `protected`, `impl`, `inherits`, `base`, `needs`, `closed`, `extendable`, `singleton`, `data`, `choice`, `alias`, `given`, `generate`, `augment`, `read`, `write`, `lazy`, `late`, `constant`, `forward`, `choose`, `when`, `if`, `elif`, `else`, `while`, `for`, `in`, `next`, `stop`, `return`, `self`, `with`, `defer`, `async`, `await`, `task`, `guard`, `try`, `handle`, `finally`, `raise`, `expect`, `require`, `ensure`, `external`, `bridge`, `annotation`, `target`, `legacy`, `test`, `before`, `after`, `init`, `override`, `true`, `false`, `null`, `and`, `or`, `not`, `as`, `is`, and the built-in type names in Section 3. Java keywords are not Kalm keywords unless listed here, so Java identifiers remain usable in interop.

Text literals use double quotes. Supported escapes are `\\`, `\"`, `\n`, `\r`, `\t`, and `\u{HEX}` for a Unicode scalar value. Interpolated text uses `@"..."`; `$name` inserts a name and `${expression}` inserts an expression. A literal dollar sign is `$$`.

Decimal integer literals have type `Int`, or `Long` with an `L` suffix, and must fit their signed 32-bit or 64-bit range. Floating literals default to `Double`; an `f`/`F` suffix gives `Float`. Numeric separators and hexadecimal/octal forms are deferred. A character literal uses single quotes and contains exactly one UTF-16 code unit.

```kalm
fix note = @"Today is ${LocalDate.now()}";
fix quoted = "She said: \"hello\"";
```

3. Types, values, and Java mapping

Kalm's built-in types map directly to Java's primitive and reference model. Java signatures remain unchanged at the boundary.

Kalm type	Java representation	Meaning
`Unit`	`void` return only	No result
`Bool`	`boolean` / `Boolean` boxed	`true` or `false`
`Byte`	`byte` / `Byte` boxed	Signed 8-bit integer
`Short`	`short` / `Short` boxed	Signed 16-bit integer
`Int`	`int` / `Integer` boxed	Signed 32-bit integer
`Long`	`long` / `Long` boxed	Signed 64-bit integer
`Float`	`float` / `Float` boxed	IEEE 754 binary32
`Double`	`double` / `Double` boxed	IEEE 754 binary64
`Char`	`char` / `Character` boxed	One UTF-16 code unit
`Text`	`java.lang.String`	Immutable Unicode text
`Any`	`java.lang.Object`	Explicit Java-compatible top type
`Array<T>`	`T[]`	Fixed-size reference array; primitive components boxed
`Task<T>`	`kalm.runtime.Task<T>`, a Java subtype of `CompletionStage<T>`	Asynchronous result handle
`BoolArray` ... `CharArray`	Matching Java primitive array	Fixed-size primitive arrays, including `ShortArray` for Java `short[]`
`List<T>`	`java.util.List<T>`	Ordered mutable list
`Map<K,V>`	`java.util.Map<K,V>`	Mutable map with non-null keys
`T?`	Boxed/reference type	Nullable value; `null` is its null value

`Unit` is allowed only as a function or method result. Nullable primitive types use Java wrapper classes. List literals `[a, b]` create mutable `java.util.ArrayList` values; map literals create insertion-ordered `java.util.LinkedHashMap` values. Generic arguments are invariant in Kalm; Java generic declarations retain their bounds and wildcard information during interop.

Arithmetic requires matching types, except `Byte`, `Short`, and `Char` promote to `Int` as in Java. There are no implicit narrowing conversions. Explicit conversions are `toByte()`, `toShort()`, `toInt()`, `toLong()`, `toFloat()`, `toDouble()`, and `toChar()`. Integral narrowing is checked and throws `ArithmeticException`; floating-to-integral conversion truncates toward zero and rejects NaN, infinity, and out-of-range values. Integer overflow wraps at the result width. Integer division truncates toward zero.

`true` and `false` are Boolean literals. `null` requires a nullable target or `Any`. A nullable receiver uses safe navigation (`value?.name`). Safe indexing uses `value?[index]`; it evaluates the receiver once and, if it is null, returns null without evaluating the index. The receiver must be nullable. It is supported for arrays, `Text`, `List`, and `Map`; the result is nullable, and a missing map key also yields null. The null-coalescing operator `left ?? right` evaluates `left` once and evaluates `right` only when `left` is null. Its left operand must be nullable, and the result type is the common type of the non-null left type and the right type (and is nullable when the right type is nullable). `??=` assigns a fallback only when a mutable nullable local or field is null; the target and fallback are each evaluated at most once. As an expression, `??=` yields the existing non-null value or the newly assigned fallback. These are syntax-level operations and add no runtime library dependency. Inside `if (value != null) { ... }`, a stable `fix` local or field is refined to non-null. Mutable `var` values are not smart-cast. Values of type `Any` require `as Type` before specialized use; failed casts raise `ClassCastException`. `is Type` checks and refines a stable value.

Local `fix`/`var` types may be inferred when initialized. Parameters, fields, and public function results require explicit types. `Task<T>` is a public Kalm runtime interface that extends `java.util.concurrent.CompletionStage<T>`. Generated Java methods keep `Task<T>` as their return type, and Java callers can use or assign it as a `CompletionStage<T>`. `await` accepts either `Task<T>` or any Java `CompletionStage<T>`. An empty collection or `null` requires a target type. Kalm lambdas are context-typed and may target Kalm or Java single-abstract-method contracts (SAMs); standalone untyped function values are deferred.

4. Bindings, expressions, and functions

`fix` declares an immutable binding; `var` declares a mutable one. In type or member modifier position, `fix` means that a class cannot be subclassed or a member cannot be overridden. Fields use the same binding words. Definite assignment is checked at compile time.

KALM

```
fix startValue: Int = 3;
var score = startValue + 4;
score += 1;

fun square(value: Int) -> Int = value * value;
fun greet(name: Text) -> Text {
    return @"Hey, $name!";
}
```

`fun name(...)` declares a named function or method; `fun item => expression` introduces a context-typed lambda expression. Named functions are allowed in declaration positions and use `fun name(...)`; lambda expressions appear in expression positions and use `fun parameters => ...` or `fun parameters { ... }`. Function parameters are immutable. A block-bodied function returning a value must `return` a value on every path. A function without a result arrow returns `Unit` if block-bodied. Expression-bodied functions use `= expression`. A bare `return` is valid only in a `Unit` function. Default/named arguments and Kalm function overload declarations are deferred.

Operators evaluate left to right. `and` and `or` short-circuit. Equality uses primitive value equality, text value equality, and Java `equals` for other references. Relational comparisons support numbers, `Char`, and `Text` in UTF-16 order. `+` adds numbers or joins two `Text` values. Operators `&&`, `||`, and `!` are accepted as aliases for `and`, `or`, and `not`. Assignment targets must be mutable Kalm bindings/fields or assignable Java fields. List and array indexes use Java bounds behavior. Safe navigation evaluates its receiver once and skips arguments when the receiver is `null`.

Precedence high to low	Operators	Associativity
Postfix	call, member `.`, safe member `?.`, safe index `?[...]`, index `[]`	left
Unary	`not`, `!`, unary `+`, unary `-`	right
Multiplicative	`*`, `/`, `%`	left
Additive	`+`, `-`	left
Relational/type	`<`, `<=`, `>`, `>=`, `is`, `as`	non-associative
Equality	`==`, `!=`	non-associative
Logical	`and`, `or`	left, short-circuit
Coalescing	`??`	right
Assignment	`=`, `+=`, `-=`, `*=`, `/=`, `%=`, `??=`	right

A method that returns an overloaded Java call uses Java overload resolution. If the target remains ambiguous, Kalm reports the candidate signatures and asks for a type annotation or cast.

Kalm adds small null-flow conveniences for cases developers regularly discuss as awkward in Java: coalescing a nullable value, safe indexing, and assigning a fallback. They lower to conditional Java code and reuse the existing nullable type checker and safe-member code generation.

KALM

```
var names: List<Text>? = findNames();
fix firstName = names?[0] ?? "Guest";

var title: Text? = findTitle();
title ??= "Untitled";
```

5. Flow, lambdas, and errors

`if`, `elif`, and `else` branch on `Bool` conditions; `while` repeats as long as its condition is true. `for item in items` accepts `Iterable<T>` or an array and creates an immutable loop variable. `stop` exits the innermost loop; `next` advances to its next iteration.

KALM

```
if (score > 0) {
    sayLine("Ready.");
```

```kalm
} elif (score == 0) {
    sayLine("Waiting for input.");
} else {
    sayLine("Let's reset.");
}

while (score < 10) {
    score += 1;
}

for (name in names) {
    sayLine(name);
}
```

Java SAM lambdas use a lightweight Kalm form. Their target and parameter types come from the selected method signature, and captures follow Java's effectively-final rule.

```kalm
fix loud: java.util.function.Function<Text, Text> = fun name => name.toUpperCase();
fix loudNames = names.stream().map(fun name => name.toUpperCase()).toList();
```

A block lambda uses braces: `fun item { ... }`; a value-returning lambda uses `return` to provide its result. The compiler rejects an ambiguous SAM overload unless a target type or cast resolves it. Named functions use `fun name(...)`; lambdas use `fun parameters => expression` or a braced lambda body.

Kalm uses JVM exceptions. `try`, `handle`, `finally`, and `raise` map directly to Java. A `handle` parameter must have a Java exception subtype. A `try` block must have at least one `handle` or a `finally` clause. Checked Java exceptions must be handled at Kalm call sites; ordinary Kalm functions do not declare `throws` clauses. When an `override fun`, accessor, `forward`, or `external fun` corresponds to a Java method with checked exceptions, generated Java preserves the compatible Java `throws` clause. `raise` may rethrow unchecked exceptions freely; a checked exception may be raised only within a `try` whose `handle` clause catches it. A checked exception cannot escape an ordinary Kalm function. Other unhandled exceptions behave as they do in Java.

```kalm
try {
    sayLine(Files.readString(Path.of("greeting.txt")));
} handle (error: java.io.IOException) {
    sayLine(error.getMessage());
} finally {
    sayLine("done");
}
```

Matching and checks

`choice` declares a finite set of named variants and compiles to a Java enum; variants carry no payload or type parameters in 1.x. `choose` evaluates a value once and selects a `when` branch. It must be exhaustive for a `choice` and for a `closed` hierarchy; `_` is the explicit catch-all branch.

```kalm
choice Status {
    Ready;
    Failed;
}

fix message = choose status {
    when Status.Ready => "Ready to continue";
    when Status.Failed => "Please retry";
};
```

`expect condition` raises `AssertionError` when false. `require condition` validates a function precondition and raises `IllegalArgumentException`; `ensure condition` validates a postcondition and raises `IllegalStateException`. Optional messages follow a comma. These checks are always active and are not removed by build settings.

Resource and scope management

`with name = expression { ... }` requires an `AutoCloseable` result and closes it when the block exits, including exceptional exits. Multiple resources close in reverse declaration order. `defer { ... }` registers a cleanup block for the end of the current function or block; deferred blocks run in reverse registration order on normal return or exception. Control transfer (`return`, `stop`, or `next`) is not allowed inside a deferred block.

```kalm
with reader = Files.newBufferedReader(path) {
    return reader.readLine();
}

defer {
    transaction.rollbackIfOpen();
}
```

Tasks and synchronization

`async fun` runs a function body on the Kalm virtual-thread executor and returns `Task<T>`, whose public Java interface extends `java.util.concurrent.CompletionStage<T>`. Its declared result type `T` is the value produced by the body. `await task` waits for completion and returns its value; it is permitted only inside an `async fun` or a `task { ... }` block. Awaiting rethrows runtime exceptions and errors, and wraps checked failures in `TaskFailureException` with the original cause. Interruption restores the interrupt flag and raises `TaskInterruptedException`.

`task { ... }` starts an asynchronous block and returns `Task<T>`. Its body follows ordinary function block rules: it must return a value on every path, using `return` for a value result. The runtime uses Java virtual threads for blocking-friendly work; CPU-heavy parallel work should use Java's executor APIs directly. A `guard lock { ... }` block uses the Java monitor for `lock`; `await` is prohibited inside a guard so a task never suspends while holding a monitor. Each `async fun` or `task` uses a virtual thread; the runtime owns and closes its executor during application shutdown.

```kalm
async fun loadProfile(id: Text) -> Profile {
    return await profileClient.fetch(id);
```

```kalm
}

fix refresh = task {
    return await loadProfile("42");
};

guard cache {
    cache.put("profile", refresh);
}
```

The public `kalm.runtime.Task<T>` interface extends `CompletionStage<T>` and adds `cancel()` and `isCancelled()`. Its runtime implementation delegates completion operations to a `CompletableFuture<T>` and tracks its virtual thread so `cancel()` can interrupt it when possible. Java methods that return `Task<T>` remain directly usable as `CompletionStage<T>`; `await` also accepts ordinary Java completion stages.

6. Classes, contracts, and visibility

`class` declares a JVM class. `contract` declares a JVM interface. `init` declares a constructor. If none is present, the compiler follows the default-constructor rule below. `self` refers to the current instance. Calling `ClassName(arguments)` constructs Kalm and Java classes. Constructor overload declarations inside a Kalm class are deferred; use named factory functions. Fields initialize in declaration order. A `fix` field is assigned exactly once in its declaration or constructor. Every non-null reference field must be initialized or definitely assigned by every constructor; nullable references default to `null`, and uninitialized primitive `var` fields use their Java zero value. A no-argument constructor is generated only when all required fields can be initialized without parameters; otherwise a missing constructor is a compile error.

```kalm
pub class Person {
    fix name: Text;
    var mood: Text;

    init(name: Text) {
        self.name = name;
        self.mood = "easy";
    }

    fun hello() -> Text = @"Hi, I'm $name!";
}

contract Greeter {
    fun greet(name: Text) -> Text;
}

class FriendlyGreeter impl Greeter {
    override fun greet(name: Text) -> Text = @"Hey, $name. Glad you're here.";
}
```

Kalm classes compile to Java classes and contracts compile to Java interfaces. A class may `inherits` one `base`, `closed`, or `extendable` Kalm class (only an explicitly permitted child may inherit a `closed` class), or any accessible non-final Java class. It may `impl` one or more contracts or Java interfaces.

A `base class` is abstract and inheritable. Only a `base class` may declare `needs fun` members without bodies; concrete subclasses must implement them. A `needs fun` declaration is an abstract requirement and is implicitly overridable until implemented. A concrete implementation must use `override fun`; it satisfies the requirement even though ordinary methods are closed to overriding by default. The implementation is itself closed unless marked `extendable`. Contract signatures are also abstract requirements: implementing a Kalm contract or Java interface requires `override fun`, and the resulting method is closed unless marked `extendable`.

A `closed class` must declare every allowed direct child as a nested declaration; the compiler emits a Java sealed hierarchy with an explicit permits list. Nested Kalm classes are static nested Java types; inner classes that capture an enclosing instance are deferred. In a closed hierarchy, a direct child with no modifier or `fix` is Java `final`; an `extendable` child is Java `non-sealed`; and a `closed` child is Java `sealed` and declares its own permits list. A `base` child is abstract and is also emitted as `non-sealed` unless combined with `closed`.

Kalm classes and methods are closed to inheritance or overriding by default. `fix` maps to Java `final`; `extendable` leaves a class or method open; `base` maps to `abstract`; and `closed` maps to Java `sealed`. `base closed` is allowed. `fix` cannot be combined with `base`, `closed`, or `extendable`; `closed` cannot be combined with `extendable`. An override may be marked `fix` to close that method again. `override fun` must match the inherited signature after generic substitution.

`data` declares an immutable Java record with components, accessors, and Java record value equality. `choice` declares a finite set of named variants and compiles to a Java enum; variants carry no payload or type parameters in 1.x. `singleton` declares one globally accessible instance, generated as a final Java class with a private constructor and `INSTANCE` field. `alias Name = Type` creates a compile-time type alias and no new JVM type. `given` adds bounds to type parameters, for example `fun sort<T>(items: List<T>) -> List<T> given T: Comparable<T>`. In 1.x the clause is required only when a type parameter has a bound.

`generate Display` overrides a data record's generated `toString()` with a stable `TypeName[field=value, ...]` form, using component declaration order and Kalm text formatting for each value. Java record equality and hashing remain those provided by the record model; they are not separately generated. `augment Type { ... }` declares Kalm-only extension functions. Inside an augment function, `self` is the receiver value; calls are rewritten to generated static helpers whose first parameter is that receiver, and do not add methods to the Java class. `bridge augment` also emits a public Java helper facade for those functions. `forward Contract to field` generates contract methods that delegate to that field; the field must implement the contract. Generated forwarding methods preserve the Java contract's checked-exception declarations.

KALM

```
use java.io.Closeable;
use java.io.InputStream;

pub data User(name: Text, id: Long) generate Display;

alias UserId = Long;

augment Text {
    fun initials() -> Text = self.substring(0, 1);
}

class FileStore impl Closeable {
    fix stream: InputStream;

    init(stream: InputStream) {
```

```kalm
        self.stream = stream;
    }

    forward Closeable to stream;
}
```

`read` and `write` declare property accessors. Kalm member access rewrites to the accessor; Java receives conventional `getX`/`setX` methods. `lazy fix` declares a read-only field evaluated once on first access and safely publishes the cached value. The initializer runs at most once, including when it throws; subsequent reads rethrow the same failure. `late var` is a non-null reference field assigned after construction; reads from both Kalm and Java use a runtime guard that throws `IllegalStateException` until assignment. Setters reject null, and subsequent assignments are allowed. No compile-time initialization protocol is promised for `late` fields. `constant` declares a public static final Java compile-time constant restricted to primitive or `Text` literal values; the compiler emits a Java constant variable where Java permits it. Constants are implicitly `pub` and top-level in 1.x. `legacy` marks a declaration deprecated in Kalm diagnostics and emits Java `@Deprecated`.

```kalm
use java.time.ZoneId;

class Profile {
    var first: Text;
    var last: Text;

    init(firstName: Text, lastName: Text) {
        first = firstName;
        last = lastName;
    }

    read fullName: Text = @"$first $last";
    write displayName(value: Text) {
        fix pieces = value.split(" ");
        first = pieces[0];
        last = pieces[1];
    }

    lazy fix zone: ZoneId = ZoneId.systemDefault();
}

constant DEFAULT_LIMIT: Int = 100;
```

Visibility words map to Java access as closely as Java permits:

Kalm word	Meaning
`pub`	Public to Java and other modules
`internal`	Current Kalm module; enforced by Kalm compilation. Java callers may see generated public members because JVM bytecode has no Kalm-module visibility.

`private`	Source-file private at top level and class-private for members. Java has no private top-level declarations, so the compiler enforces file privacy and emits package-private Java declarations; same-package Java source can therefore access those generated declarations.
`protected`	Java protected visibility for class members
omitted	Top-level declarations are `internal`; fields are `private`; methods and constructors are `pub`

Static declarations inside Kalm classes are deferred in 1.x. Java static members remain directly accessible.

7. Packages and Java interoperability

`space` gives a Java-compatible package name. `use` imports a Java or Kalm type; wildcard uses are supported. Name resolution checks local declarations, same package, explicit uses, wildcard uses, then `java.lang`. Ambiguity is an error and can be resolved by a qualified name.

Kalm can use public Java classes, constructors, fields, methods, enums, arrays, and generic types found on the compile classpath or readable module path. Kalm primitive names map to matching Java primitives. Calls follow Java visibility, overload resolution, primitive widening, boxing/unboxing, and varargs rules. Narrowing needs an explicit checked conversion. Java annotations describing nullability are honored; unannotated Java reference types are platform types and produce a warning on assignment to a non-null Kalm type.

Java sources and Kalm sources in the same module compile together. The compiler first emits analysis-only Java signature stubs for Kalm declarations, asks the configured `javac` to analyze Java sources plus those stubs, type-checks Kalm against the resulting Java language model, generates Java source, and finally compiles handwritten and generated Java sources together. Stubs never enter the packaged artifact. Java can call `pub` Kalm facade functions (for example, `GreetingApiKalm.greeting("Tim")`), and Kalm can call Java classes in that same source set. Annotation processors that generate types needed during Kalm analysis must run in an earlier generated-source step; processors that rewrite Java syntax trees are outside the 1.x contract.

KALM

```
space studio.greetings;
use java.nio.file.Files;
use java.nio.file.Path;

pub fun readGreeting() -> Text {
    try {
        return Files.readString(Path.of("greeting.txt"));
    } handle (error: java.io.IOException) {
        return "Could not read greeting";
    }
}
```

`external fun` declares a Kalm-named wrapper around a public Java static method. Its right-hand qualified name is resolved against the project's Java model, and its signature must be invocation-compatible with the target. Java libraries themselves remain directly usable without wrappers.

KALM

```
external fun parseCount(value: Text) -> Int = java.lang.Integer.parseInt;
```

`bridge augment` emits a Java helper facade for Kalm extension functions. An ordinary `augment` is usable only from Kalm; a bridged one also exposes static methods such as `TextExtensions.initials(value)` for Java callers.

`annotation` declares a Java annotation type. `target` lists where it may appear (`type`, `function`, `field`, `property`, or `parameter`). Java has no `ElementType.PROPERTY`: a Kalm `property` target maps to `ElementType.METHOD` and is emitted on the generated getter; use `field` to annotate the backing field. Other targets map to the matching Java `ElementType`. Annotation members use Java's supported annotation value types, and generated annotations default to `RetentionPolicy.CLASS`; Java meta-annotations such as `@Retention`, `@Target`, and `@Repeatable` may configure them. `@Name` or `@Name(arguments)` applies an annotation to a declaration. Parameters use `@Name parameter: Type`; type uses use `parameter: @Name Type`, including nested generic and array types. Type parameters and record components accept annotations. The target shorthand also supports `constructor`, `localVariable`, `annotationType`, `typeUse`, `typeParameter`, and `recordComponent`; conflicting shorthand and explicit `@Target` are rejected. Arguments must be Java annotation constant values, enum constants, class literals, arrays, or nested annotations; positional syntax is allowed only for a single member named `value`. `legacy` emits `@Deprecated` and a Kalm compiler warning at use sites.

Kalm tests are declared only under `src/test/kalm`. A `test "description" { ... }` block is one test case; failed `expect` checks fail that test and are reported by the runner. File-level `before { ... }` and `after { ... }` hooks run before and after each test in that file; `after` runs even when a test fails. The test runtime reports failures through Maven/Gradle test tasks and can also run through `kalm test`. `expect` is always active inside tests.

KALM

```
before {
    TestDatabase.openShared();
}

test "a user can be loaded" {
    fix user = TestSupport.repository().find("42");
    expect user.name == "Ari";
}

after {
    TestDatabase.closeShared();
}
```

8. Standard library

The separate `kalm-stdlib` jar (JPMS module `kalm.stdlib`) provides the automatically imported prelude and ordinary convenience APIs. Applications declare `dev.kalm:kalm-stdlib`, which depends transitively on `kalm-runtime`. Compiler, CLI, server, and editor distributions bundle both jars. Prelude functions are resolved from library signatures, using the same overload and generic rules as other Java methods. User declarations take precedence over the prelude.

The `kalm-runtime` jar contains language support only, including `Task<T>`, `TaskFailureException`, and `TaskInterruptedException`, and has JPMS module name `kalm.runtime`. It does not replace Java collections, I/O, time, math, or concurrency APIs. The prelude is automatically available.

Kalm API	Behavior

`say(value)`, `sayLine(value)`	Write formatted value, optionally followed by the platform newline
`len(value)`	Length/size of `Text`, arrays, `List`, or `Map`
`arrayOf(values...)`	Construct a reference array when component type is reifiable
`boolArrayOf` ... `charArrayOf`	Construct the matching Java primitive arrays
`range(end)`, `range(start, end)`	Lazy `Iterable<Int>`; exclusive end
`exit(code)`	Delegate to `System.exit`
`abs`, `min`, `max`	Numeric overloads for `Int`, `Long`, `Float`, and `Double`
`sqrt`, `pow`, `round`	Delegate to `java.lang.Math`

Formatting uses Java-compatible `null`, `true`, and `false` spellings and formats other values through `String.valueOf`. Interpolation uses the same formatting. `upper()` and `lower()` use locale-independent `Locale.ROOT`. Lists and maps are Java mutable collections. Java APIs such as `Objects`, `Collections`, `Files`, and `Math` remain directly importable.

The stable runtime methods are `kalm.runtime.Kalm.say(Object)`, `sayLine(Object)`, `text(Object)`, `range(int)`, and `range(int,int)` with Java-compatible static signatures. Primitive-array helpers use corresponding Java primitive array types.

9. Compiler and Java 25 backend

The compiler is a Java application with a reusable compiler-core API. It requires a full JDK 25+ with `javac`; a JRE is insufficient. The reference backend uses documented APIs only: `javax.tools.JavaCompiler`, `com.sun.source.util.JavacTask`, and `javax.lang.model`. It must not depend on `com.sun.tools.javac.*` internal packages or preview features.

Each compilation runs four phases:

1. Lex/parse Kalm and emit temporary Java signature stubs for public and module-visible Kalm declarations.
2. Parse and analyze handwritten Java plus stubs with `javac`, using the project's source roots, classpath, module path, and selected release.
3. Type-check Kalm against Java symbols, then emit real Java source.
4. Compile handwritten and generated Java together to ordinary class files using `javac --release N`; Kalm 1.x rejects releases below 25.

The result runs on a standard JVM, with no custom VM, classloader, or bytecode instruction set. Generated source is retained for debugging, with Kalm-to-Java source mapping for diagnostics and stack traces.

Required CLI:

```text
kalm --version
kalm check [options] <sources or source directories>
kalm compile [options] <sources or source directories>
kalm run [options] <sources or source directories> [-- program-args]
kalm test [options] <test sources or source directories>
```

Options: `--classpath`, `--module-path`, `--release` (default 25; minimum 25), `--main` (required by `run` only if multiple entry points exist), `--output`, `--generated-sources`, `--encoding`, `--diagnostic-format human|json`, and `--debug`. Exit codes: 0 success, 1 source/type error, 2 command/configuration error, 3 compiler internal error. Internal errors show a report ID; a raw Java stack trace requires `--debug`.

Generated sources default to `target/generated-sources/kalm/` under Maven or `build/generated/sources/kalm/` under Gradle. Classes use the normal host build output. Caches stay under the project build directory. `check` type-checks without class generation; `compile` emits Java and classes; `run` compiles and starts the selected Kalm entry point.

10. Maven and Gradle

The compiler core, runtime, Maven plugin, Gradle plugin, language server, VS Code extension, and IntelliJ plugin are separate versioned deliverables. Artifact coordinates and plugin IDs are finalized before the first public release and are versioned together. Dependencies are ordinary Maven/Gradle Java dependencies; Kalm resolves them from the same project classpath/module path.

Maven

Default roots are `src/main/kalm` and `src/test/kalm`, alongside standard Java roots. The plugin binds `kalm:compile` to `generate-sources` and `kalm:testCompile` to `process-test-sources`, registers generated Java roots, reads Maven dependencies and `maven.compiler.release`, supports incremental builds, and fails on Kalm errors. The later test phase lets `kalm:testCompile` use compiled main classes. It analyzes Java test sources against Kalm test signature stubs and emits generated test Java; the normal Maven test compiler then compiles handwritten and generated test Java together. `kalm:compile` analyzes main Java sources against Kalm signature stubs and emits generated Java; the normal Maven compiler then compiles handwritten and generated main Java together. The plugin binds `kalm:test` to Maven's `test` phase. It runs Kalm tests, emits Surefire-compatible XML under `target/surefire-reports/`, and fails the build when a test fails. The selected toolchain and release must be at least 25. The runtime is an ordinary dependency:

```xml
<dependency>
  <groupId>dev.kalm</groupId>
  <artifactId>kalm-stdlib</artifactId>
  <version>${kalm.version}</version>
</dependency>
```

Source roots, generated source directory, release, compiler arguments, includes, and excludes are configurable. The plugin must not change unrelated Java compiler settings. A named JPMS application adds `requires kalm.runtime;` to its `module-info.java`.

Gradle

The plugin uses `src/main/kalm` and `src/test/kalm` by default, integrated with Java source sets. `compileKalm` analyzes Java main sources against Kalm signature stubs and emits generated Java; `compileJava` then compiles handwritten and generated Java together, avoiding a task dependency cycle. After `compileJava`, `compileTestKalm` analyzes Java test sources against Kalm test signature stubs and emits generated test Java; `compileTestJava` compiles handwritten and generated test Java together. These cacheable tasks declare inputs/outputs for up-to-date checks, use the existing Java compile/runtime classpaths, and add generated Java to the corresponding source sets. A Kalm test execution task is wired into `check`/`test`.

The Kalm test task emits Gradle-compatible XML under `build/test-results/` and HTML report data, and fails `check` on a test failure. Toolchains and release are 25+. The runtime is a normal dependency:

KOTLIN
```
dependencies {
    implementation("dev.kalm:kalm-stdlib:<version>")
}
```

Both plugins must compile mixed Java/Kalm modules and test source sets, fix Java call Kalm and Kalm call Java within one module, preserve ordinary dependency resolution, and avoid a Java/Kalm task cycle using the joint analysis-and-final-compilation phases above.

11. Editor support

Editor support is part of the release contract. One Kalm Language Server Protocol implementation supplies syntax and semantic diagnostics, completion, hover, go-to-definition, references, document symbols, rename, formatting, and semantic tokens. It uses the project compiler version and resolved Java classpath where available.

- **VS Code:** extension recognizes `.kalm` and language ID `kalm`; starts or downloads a compatible language server; integrates Maven/Gradle classpaths; provides highlighting, snippets, diagnostics, completion, hover, navigation, rename, and formatting; supports compiler path/version settings.
- **IntelliJ IDEA:** plugin recognizes `.kalm`; provides highlighting, diagnostics, completion, navigation, rename, and formatting; imports Kalm roots/dependencies in Maven/Gradle projects; may embed the shared language server; does not require Ultimate-only features.

Editor packages share fixtures for parsing, highlighting, diagnostics, and formatting. Formatting normalizes related sibling blocks to the first block's brace/colon style, while nested groups choose independently and empty blocks remain `{}`. It preserves explicit semicolons, is deterministic and idempotent, and checks without writing unless `--fix` is supplied; `1.x` exposes indentation width (for visual indentation inside braces) and line length settings.

12. Core grammar

This grammar is normative. Semicolons or logical newlines terminate simple statements; braces or colon suites delimit blocks, as defined in Section 2. A semantic modifier check rejects duplicate modifiers and the invalid combinations described in Section 6. The parser rejects missing statement terminators, unmatched braces, and malformed statements rather than guessing. The precedence productions below resolve the shorthand `Expr` rule used in statement productions. `??=` is a right-associative assignment operator; `??` is a right-associative coalescing operator between logical and assignment precedence.

EBNF
```
File              = [ SpaceDecl ] { UseDecl } { Declaration } EOF ;
SpaceDecl         = "space" QualifiedName Terminator ;
UseDecl           = "use" QualifiedName [ "as" Identifier | "." "*" ] Terminator ;
Declaration       = { AnnotationUse } [ Visibility ] [ "legacy" ]
                    ( TypeDecl | FunctionDecl | AliasDecl | ConstantDecl
                    | AugmentDecl | ExternalDecl | AnnotationDecl | TestDecl | HookDecl ) ;
AnnotationUse     = "@" QualifiedName [ "(" [ AnnotationArguments ] ")" ] ;
AnnotationArguments = AnnotationArgument { "," AnnotationArgument } ;
AnnotationArgument  = [ Identifier "=" ] AnnotationValue ;
```

EBNF

```
AnnotationValue     = ConstantExpr | Type "." "class" | "{" [ AnnotationValues ] "}" |
NestedAnnotation ;
AnnotationValues    = AnnotationValue { "," AnnotationValue } ;
NestedAnnotation    = "@" QualifiedName [ "(" [ AnnotationArguments ] ")" ] ;
Visibility        = "pub" | "internal" | "private" | "protected" ;
TypeDecl          = ClassDecl | ContractDecl | DataDecl | ChoiceDecl | SingletonDecl ;
TypeModifier      = "base" | "closed" | "fix" | "extendable" ;
ClassDecl         = { TypeModifier } "class" Identifier [ TypeParams ] [ InheritsClause ] [
ImplClause ] MemberBlock ;
InheritsClause    = "inherits" Type ;
ImplClause        = "impl" TypeList ;
ContractDecl      = "contract" Identifier [ TypeParams ] [ "inherits" TypeList ]
ContractBlock ;
DataDecl          = "data" Identifier [ TypeParams ] "(" [ Params ] ")" [ GenerateClause ]
Terminator ;
ChoiceDecl        = "choice" Identifier ChoiceBlock ;
ChoiceBlock       = "{" { ChoiceVariant } "}" | ":" DeclarationSuite(ChoiceVariant) ;
ChoiceVariant     = Identifier Terminator ;
SingletonDecl     = "singleton" Identifier MemberBlock ;
MemberBlock       = "{" { Member } "}" | ":" DeclarationSuite(Member) ;
Member            = { AnnotationUse } [ Visibility ] [ "legacy" ]
                    ( Field | Constructor | FunctionDecl | NeedsFunction
                    | Accessor | ForwardDecl | TypeDecl ) ;
ContractBlock     = "{" { ContractMember } "}" | ":" DeclarationSuite(ContractMember) ;
ContractMember    = { AnnotationUse } ( FunctionSignature | AccessorSignature ) ;
FunctionHead      = [ "async" ] "fun" Identifier [ TypeParams ] "(" [ Params ] ")"
                    [ "->" Type ] [ GivenClause ] ;
FunctionSignature = FunctionHead Terminator ;
NeedsFunction     = "needs" FunctionSignature ;
Constructor       = "init" "(" [ Params ] ")" Block ;
FunctionDecl      = [ "override" ] [ "async" ] [ "extendable" | "fix" ] "fun" Identifier [
TypeParams ]
                    "(" [ Params ] ")" [ "->" Type ] [ GivenClause ] FunctionBody ;
FunctionBody      = "=" Expr Terminator | Block ;
AliasDecl         = "alias" Identifier [ TypeParams ] "=" Type Terminator ;
ConstantDecl      = "constant" Identifier ":" Type "=" ConstantExpr Terminator ;
Field             = [ "lazy" ] [ "late" ] ( "fix" | "var" ) Identifier ":" Type [ "=" Expr ]
Terminator ;
Accessor          = "read" Identifier ":" Type ( "=" Expr Terminator | Block )
                    | "write" Identifier "(" Param ")" Block ;
AccessorSignature = "read" Identifier ":" Type Terminator
                    | "write" Identifier "(" Param ")" Terminator ;
ForwardDecl       = "forward" Type "to" Identifier Terminator ;
GenerateClause    = "generate" Identifier { "," Identifier } ;
GivenClause       = "given" TypeConstraint { "," TypeConstraint } ;
TypeConstraint    = TypeParam ":" Type ;
TypeList          = Type { "," Type } ;
Params            = Param { "," Param } ;
Param             = { AnnotationUse } Identifier ":" Type ;
TypeParams        = "<" TypeParam { "," TypeParam } ">" ;
TypeParam         = { AnnotationUse } Identifier ;
Type              = { AnnotationUse } NamedType [ "?" ] ;
NamedType         = QualifiedName [ "<" Type { "," Type } ">" ]
                    | "Unit" | "Bool" | "Byte" | "Short" | "Int" | "Long"
                    | "Float" | "Double" | "Char" | "Text" | "Any" | "Task" "<" Type ">"
                    | "Array" "<" Type ">" | "BoolArray" | "ByteArray" | "ShortArray"
                    | "IntArray" | "LongArray" | "FloatArray" | "DoubleArray" | "CharArray"
```

EBNF

```
                    | "List" "<" Type ">" | "Map" "<" Type "," Type ">" ;
AugmentDecl       = [ "bridge" ] "augment" Type AugmentBlock ;
AugmentBlock      = "{" { AugmentFunction } "}" | ":" DeclarationSuite(AugmentFunction) ;
AugmentFunction   = [ Visibility ] [ "legacy" ] FunctionDecl ;
ExternalDecl      = "external" "fun" Identifier [ TypeParams ] "(" [ Params ] ")"
                    [ "->" Type ] [ GivenClause ] "=" QualifiedName Terminator ;
AnnotationDecl    = "annotation" Identifier AnnotationBlock ;
AnnotationBlock   = "{" { AnnotationMember } "}" | ":" DeclarationSuite(AnnotationMember) ;
AnnotationMember  = TargetDecl | AnnotationElement ;
TargetDecl        = "target" TargetKind { "," TargetKind } Terminator ;
TargetKind        = "type" | "function" | "field" | "property" | "parameter"
                    | "constructor" | "localVariable" | "annotationType" | "typeUse"
                    | "typeParameter" | "recordComponent" ;
AnnotationElement = Identifier ":" Type [ "=" AnnotationValue ] Terminator ;
TestDecl          = "test" TextLiteral Block ;
HookDecl          = ( "before" | "after" ) Block ;
Block             = "{" { Statement } "}" | ":" Suite ;
Suite             = SimpleSuite | NEWLINE INDENT Statement { Statement } DEDENT ;
SimpleSuite       = SimpleStatement { SimpleStatement } ;
DeclarationSuite(Element) = Element { Element }
                    | NEWLINE INDENT Element { Element } DEDENT ;
SimpleStatement   = Binding | ReturnStmt | RaiseStmt | CheckStmt | StopStmt | NextStmt
                    | Expr Terminator ;
BlockEnd          = "}" | DEDENT ;
Terminator        = ";" | NEWLINE | BlockEnd | EOF ;
Statement         = Binding | IfStmt | WhileStmt | ForStmt | ChooseStmt | ReturnStmt |
RaiseStmt
                    | TryStmt | WithStmt | DeferStmt | GuardStmt | CheckStmt | StopStmt |
NextStmt
                    | Block | Expr Terminator ;
Binding           = { AnnotationUse } ( "fix" | "var" ) Identifier [ ":" Type ] "=" Expr
Terminator ;
IfStmt            = "if" Condition Block { "elif" Condition Block } [ "else" Block ] ;
Condition         = Expr | "(" Expr ")" ;
WhileStmt         = "while" Condition Block ;
ForStmt           = "for" ( "(" Identifier "in" Expr ")" | Identifier "in" Expr ) Block ;
ChooseStmt        = ChooseExpr Terminator ;
ChooseExpr        = "choose" Expr ChooseBlock ;
ChooseBlock       = "{" { WhenArm } "}" | ":" DeclarationSuite(WhenArm) ;
WhenArm           = "when" Pattern "=>" ( Expr Terminator | Block ) ;
Pattern           = QualifiedName | "_" | Literal ;
ReturnStmt        = "return" [ Expr ] Terminator ;
RaiseStmt         = "raise" Expr Terminator ;
TryStmt           = "try" Block ( HandleClause { HandleClause } [ "finally" Block ]
                    | "finally" Block ) ;
HandleClause      = "handle" "(" Identifier ":" Type ")" Block ;
WithStmt          = "with" Resource { "," Resource } Block ;
Resource          = Identifier "=" Expr ;
DeferStmt         = "defer" Block ;
GuardStmt         = "guard" Expr Block ;
CheckStmt         = ( "expect" | "require" | "ensure" ) Expr [ "," Expr ] Terminator ;
StopStmt          = "stop" Terminator ;
NextStmt          = "next" Terminator ;
TaskExpr          = "task" Block ;
Lambda            = "fun" LambdaParams "=>" Expr | "fun" LambdaParams Block ;
LambdaParams      = LambdaParam { "," LambdaParam } | "(" [ LambdaParam { "," LambdaParam } ]
")" ;
```

EBNF

```
LambdaParam       = { AnnotationUse } Identifier [ ":" Type ] ;
Expr              = Assignment ;
Assignment        = Coalesce [ AssignmentOp Assignment ] ;
Coalesce          = LogicalOr [ "??" Coalesce ] ;
AssignmentOp      = "=" | "+=" | "-=" | "*=" | "/=" | "%=" | "??=" ;
LogicalOr         = LogicalAnd { ( "or" | "||" ) LogicalAnd } ;
LogicalAnd        = Equality { ( "and" | "&&" ) Equality } ;
Equality          = Relational [ ( "==" | "!=" ) Relational ] ;
Relational        = Additive [ ( "<" | "<=" | ">" | ">=" ) Additive | ( "is" | "as" ) Type ]
;
Additive          = Multiplicative { ( "+" | "-" ) Multiplicative } ;
Multiplicative    = Unary { ( "*" | "/" | "%" ) Unary } ;
Unary             = ( "not" | "!" | "+" | "-" | "await" ) Unary | Postfix ;
Postfix           = Primary { Call | MemberAccess | SafeMember | SafeIndex | Index } ;
Call              = [ TypeArgs ] "(" [ Arguments ] ")" ;
MemberAccess      = "." Identifier ;
SafeMember        = "?." Identifier ;
SafeIndex         = "?[" Expr "]" ;
Index             = "[" Expr "]" ;
Arguments         = Expr { "," Expr } ;
Primary           = Literal | Identifier | "self" | "(" Expr ")" | List | Map | Lambda |
TaskExpr | ChooseExpr ;
List              = "[" [ Arguments ] "]" ;
Map               = "{" [ MapEntry { "," MapEntry } ] "}" ;
MapEntry          = Expr ":" Expr ;
Literal           = IntegerLiteral | FloatLiteral | CharLiteral | TextLiteral |
InterpolatedText
                  | "true" | "false" | "null" ;
ConstantExpr      = Literal | QualifiedName | Unary | Additive ;
TypeArgs          = "<" Type { "," Type } ">" ;
QualifiedName     = Identifier { "." Identifier } ;
```

Calls on a name resolving to a type construct it; other calls invoke a function or method. Generic type arguments before constructor parentheses support Java calls such as `ArrayList<Text>()`. `choose` patterns match enum constants, closed-hierarchy types, literals, or `_`; component destructuring is not supported in 1.x. Braced blocks are valid only in declarations, control flow, resource/task/guard blocks, tests, and lambdas. `fix` has context-dependent but related meanings: immutable binding, non-extendable class, or non-overridable member. A value introduced by `fix` cannot be reassigned. `fun` introduces both named functions and lambdas: declarations have a name and parenthesized parameters, while lambdas use `=>` or a brace/colon body. Colon lambdas require parenthesized parameters. Expression parsing follows the precedence table in Section 4.

13. Diagnostics, conformance, and delivery gates

Diagnostics have stable codes (`KALM-1xxx` syntax, `KALM-2xxx` type/name, `KALM-3xxx` interop/build, `KALM-9xxx` internal), severity, UTF-16 source range, concise message, and optional related locations. Human and JSON formats are required. Source/type errors are always fatal.

A release candidate is conforming only if:

1. Parser fixtures accept the examples and reject malformed statements and unmatched braces with source ranges.
2. Type checks reject unknown names, mismatches, unsafe nullable access, missing returns, immutable writes, unresolved overloads, invalid safe-index receivers, invalid `??` types, and invalid `??=` targets.

3. Generated Java compiles with `--release 25` and behaves as specified.
4. Mixed Java/Kalm Maven and Gradle projects compile, test, package, and run using their normal dependencies and module paths.
5. Interop fixtures cover Java primitive signatures (`List.get(int)`), arrays, varargs, overloads, a Java SAM/stream call, Kalm implementation of a Java interface, Java calling Kalm, Kalm calling same-source Java, and a third-party library.
6. Conformance fixtures cover contract signatures and implementations, `choice` exhaustiveness, `data` records, `base`/`needs`/`inherits`/`impl`, valid and invalid class modifier combinations, `fix`/`extendable`, `singleton`, null coalescing and lazy fallback evaluation, safe-index receiver/index evaluation, accessors, `augment`, `forward`, `external`, annotation targets and values, `legacy`, `expect`/`require`/`ensure`, `with`/`defer`, `async`/`await`/`task`/`guard`, Java `CompletionStage` interop, checked-exception forwarding, tests and hooks.
7. CLI, language server, VS Code, and IntelliJ pass shared parse/diagnostic/format fixtures.
8. Clean builds are reproducible; incremental builds rebuild changed inputs and dependents.

14. Implementation plan

All deliverables use Java and share the compiler core. Build a JDK 25 multi-module repository.

1. **Language core:** lexer and layout parsing for optional semicolons and mixed brace/colon blocks, parser, AST, symbol table, type checker, stable diagnostics, and `kalm check`. Shared editor conformance includes scoped completion, dependency packages, annotation argument keys, and incomplete-program recovery.
2. **JVM slice:** Java source generator, source maps, Java Compiler API invocation, runtime jar, and `compile`/`run`; verify with JDK 25.
3. **Interop:** Java signature stubs and joint compilation tests in both call directions.
4. **Build integrations:** Maven/Gradle plugins, test roots, dependency/classpath wiring, incremental inputs and outputs.
5. **Editor:** LSP first, then VS Code and IntelliJ packages using shared conformance fixtures.
6. **1.1 gate:** publish all artifacts, document install/build/run, ship a mixed-language sample, and pass Section 13.

Each milestone produces a runnable artifact and CI fixtures. Before 1.0, breaking changes require migration notes; after 1.0, source and public interop compatibility follow semantic versioning.

15. Deferred from 1.x

Static declarations inside Kalm classes, operator overloading, named/default arguments, destructuring, reflection APIs, a Kalm package manager, a source-level debugger, standalone untyped function values, and Kalm function overload declarations are deferred. Java equivalents remain usable through interop where feasible. Inheritance, annotations, async tasks, augment extensions, and `choose`/`when` matching are part of 1.x as specified above.

The review found recurring developer complaints about null checks and the lack of named call arguments. Kalm's low-cost additions target nullable-flow boilerplate: `??`, `??=`, and `?[...]` lower to conditional Java and reuse the existing nullable type checker and safe-member code generation. Named arguments remain deferred because Java interop cannot reliably recover parameter names from arbitrary Java dependencies, and supporting them across compiled Kalm modules needs a deliberate metadata and call-resolution design. Checked-exception propagation remains as already specified; changing it would affect function types and interop contracts, beyond a small syntax addition. The current Java SE 27 specification and JDK 27 language-change notes do not add named arguments, null coalescing, or safe indexing. These are representative public discussions, not a claim that these are the only or most frequent Java complaints.

16. First vertical slice

The first deliverable compiles this mixed project on JDK 25:

- A `.kalm` source with `space`, `pub fun main`, a Kalm `class`, and a call to `java.time`.
- A Java class that calls a public Kalm facade function.
- A Kalm source that calls that Java class from the same source set.
- The emitted Java source, ordinary class files, and a runnable application using `--release 25`.

Implement lexer and parser fixtures before the backend, then prove the Java Compiler API joint-compilation phases before adding Maven, Gradle, or editor features. This keeps the core buildable in Java and tests the highest-risk interoperability promise early.

References

- Java Compiler API, JDK 25
- JavacTaskand source-tree APIs, JDK 25
- Virtual threads, JDK 25
- CompletableFuture, JDK 25
- Record classes, JDK 25
- The `javac` command, JDK 25
- Java Language Specification, Java SE 25
- Java Language Specification, Java SE 27
- Significant Changes in JDK 27
- Java Virtual Machine Specification, Java SE 25
- Stack Overflow: Java named method arguments
- Stack Overflow: avoiding null checks in Java